

Calidad

El software se puede definir como un conjunto de programas intangibles encargados de recibir órdenes, procesar datos y reflejar resultados.

La calidad en el software se relaciona con el cumplimiento de los requerimientos formulados por el usuario, de tal forma que, si un programa no cumple con alguno de estos requerimientos, es un software de baja calidad.

También existen condiciones implícitas que el software debe cumplir como eficiencia, seguridad, integridad, consistencia, cumplimiento de estándares especificados, etc.

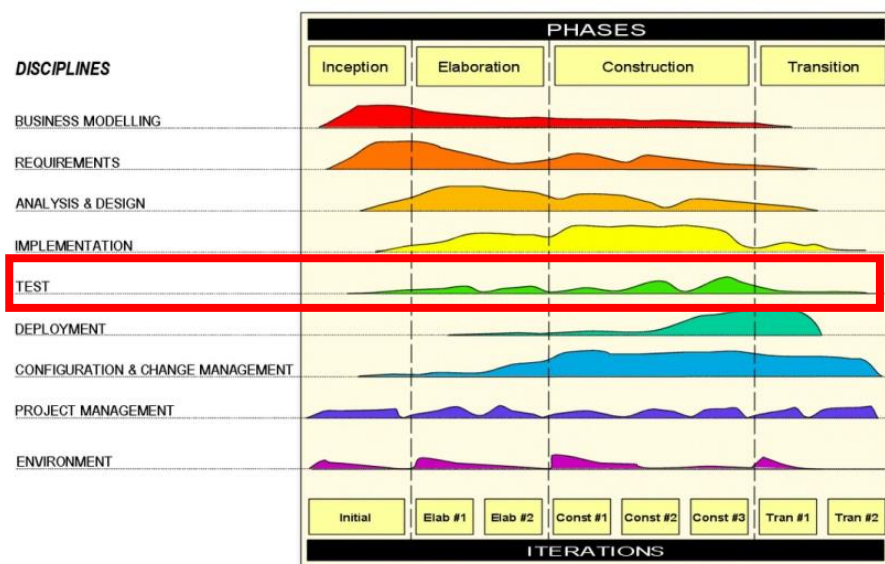
Todos los participantes del proyecto son responsables de la calidad

Proyectos

Es un esfuerzo temporal que se lleva a cabo para crear un producto, servicio o resultado único.

Agrupación de procesos de dirección:

- Iniciación: Definir el objetivo y los recursos.
- Planificación: Establecer cronogramas, presupuesto y gestión de riesgos.
- Ejecución: Realizar el trabajo necesario para cumplir con los objetivos.
- Seguimiento y control: Monitorear el progreso y realizar ajustes si es necesario.
- Cierre: Finalizar el proyecto y entregar los resultados.



Gestión de proyectos

La gestión de proyectos es la aplicación de conocimiento, habilidades, herramientas y técnicas a las actividades de un proyecto para satisfacer los requisitos del mismo.

Gestor del proyecto (administrador)

- Planear
- Organizar
- Asignar personal
- Dirigir
- Monitorear
- Controlar y ejecutar acciones correctivas
- Innovar
- Representar a clientes y usuario

Retos:

- Enfrentar fechas límites
- Enfrentar limitaciones de recursos
- Comunicar efectivamente
- Conseguir compromisos
- Establecer hitos medibles
- Enfrentar cambios
- Enfrentar conflictos
- Negociar con vendedores y subcontratistas

¿Por qué es importante la gestión de riesgos en la validación y verificación de software?

Gestión de riesgos

Riesgo es una medida de la magnitud de los daños frente a una situación peligrosa

Tarea del director del proyecto. Anticipar los riesgos que podrían afectar la programación del proyecto o la calidad del software a desarrollar y emprender acciones para evitar estos riesgos.

La gestión del riesgo es documentar el análisis de los riesgos a lo largo del proyecto junto con el análisis de consecuencias cuando el riesgo ocurra, identificar estos y crear planes para minimizar sus efectos en el proyecto, para el software y para la organización.

La gestión del riesgo se refiere a:

1. **Documentar los riesgos:** Identificar y registrar los posibles riesgos a lo largo del proyecto, manteniendo un análisis claro y actualizado.
2. **Analizar las consecuencias:** Evaluar qué podría suceder si un riesgo se materializa, analizando su impacto en el proyecto, el software y la organización.
3. **Planificar respuestas:** Diseñar estrategias para:
 - Reducir o eliminar los riesgos.
 - Mitigar su impacto en caso de que ocurran.
 - Crear planes de contingencia para garantizar que el proyecto pueda continuar.

Enuncie y explique las categorías de riesgos

Categorías:

- **Riesgos de proyecto:** estos afectan la calendarización o los recursos del proyecto. Ej: pérdida de un diseñador experimentado
- **Riesgos del producto:** Estos afectan a la calidad o al rendimiento del software que se está desarrollando. Ej: el rendimiento en un componente que hemos comprado sea menor que el esperado.
- **Riesgo del negocio:** Afecta a la organización que desarrolla o suministra el software. Ej: que un competidor introduzca un nuevo producto.

Fuentes de riesgo:

- Requisitos ambiguos
- Estimaciones de tiempo y recursos
- Habilidades personales y grupales
- Cambios en los requisitos

Proceso de gestión de riesgos:

- Identificación de riesgos
- Análisis de riesgos
- Planificación de riesgos
- Supervisión de riesgos

Es un proceso iterativo que se aplica a lo largo de todo el proyecto. Elaborando un plan inicial, se supervisa en cuanto surja más información acerca de riesgos, se analiza nuevamente.

Estrategias frente al riesgo

Estrategias reactivas:

- Método:
 - Evaluar las consecuencias del riesgo cuando este ya se ha producido (ya no es un riesgo)
 - Actuar en consecuencia
- Consecuencias:
 - Apagado de incendios
 - Gabinetes en crisis
 - Se pone el proyecto en peligro

Estrategias proactivas:

- Método:
 - Evaluación previa y sistemática de riesgos
 - Evaluación de consecuencias
 - Plan de evitación y minimización de consecuencias
 - Plan de contingencia
- Consecuencias:
 - Evasión del riesgo
 - Menor tiempo de reacción
 - Justificación frente a los superiores

Identificación de riesgos

Primera actividad de la gestión de riesgos

Comprende el descubrimiento de los posibles riesgos del proyecto

Se identifican con tormenta de ideas, experiencia, lista de posibles riesgos

Se obtiene una lista de riesgos

Grupos de riesgos:

- Genéricos: comunes a todos los proyectos
- Específicos: Implican un conocimiento profundo del proyecto

Categorías:

- **Tamaño del producto**
- **Impacto en la organización:** Se derivan del entorno organizacional donde el software se está desarrollando.
- **Tipo de cliente**
- **Definición del proceso de producción**
- **Entorno de desarrollo**

- **Tecnología:** Se derivan de las tecnologías de software o de hardware utilizadas en el sistema que se esta desarrollando.
- **Experiencia y tamaño del equipo:** riesgos asociados con las personas del equipo de desarrollo
- **Herramientas:** Se derivan de herramientas CASE y de otro software de apoyo utilizado para desarrollar el sistema
- **Requerimientos:** Se derivan de los cambios de los requerimientos del cliente y el proceso de gestionar dicho cambio
- **Estimación:** Se derivan de los estimados administrativos de las características del sistema y los recursos requeridos para construir dicho sistema.

Análisis de riesgos

Se considera por separado cada riesgo identificado y se decide acerca de la probabilidad y seriedad del mismo
No existe una forma sencilla

Valoración empírica

- Probabilidad del riesgo
 - o Muy bajo
 - o Bajo
 - o Moderado
 - o Alto
 - o Muy alto
- Severidad – impacto
 - o Insignificante
 - o Tolerable
 - o Serio
 - o Catastrófico

Se crea una tabla de riesgos donde se ordenan y filtra por probabilidad y prioridad. Se desprecian los riesgos poco probables con poco impacto

Tanto la probabilidad como el impacto cambian conforme se disponga de mayor información acerca del riesgo y los planes de gestión del mismo se implementen

La tabla de riesgos debe actualizarse durante cada iteración del proceso de riesgos.

Planificación de riesgos

El proceso de planificación de riesgos considera cada uno de los riesgos clave que han sido identificados, así como las estrategias para gestionarlo.

Estrategias:

- Prevención: reducir la aparición del riesgo
- Minimización: reducir el impacto del riesgo
- Plan de contingencia: estrategia para aplicar ante la aparición del riesgo crítico

Supervisión de riesgos

La supervisión del riesgo normalmente valora cada uno de los riesgos identificados para decidir si este es mas o menos probable y si han cambiado sus efectos.

Es un proceso continuo, y en cada revisión del progreso de gestión, cada riesgo clave debe ser considerado y analizado por separado.

Validación y Verificación

¿Qué es la validación y verificación de software?

Son actividades encaminadas a determinar si se está construyendo el producto correcto de la manera correcta.

Se utiliza para mostrar que el sistema se ajusta a su especificación y que cumple con las expectativas del cliente que lo comprará.

El objetivo es establecer la seguridad de que el sistema es lo suficientemente bueno para su uso predeterminado.

Verificación ¿Estamos construyendo el producto de la manera correcta?

La verificación implica comprobar que el software está de acuerdo con su especificación.

Es el proceso de determinar si los productos de una determinada fase del ciclo del proceso de desarrollo cumplen los requerimientos establecidos durante la fase previa.

Las actividades de verificación son desarrolladas de manera concurrente con las actividades del desarrollo de software.

La verificación se da en torno a tres procesos básicos:

- Inspección: es una revisión técnica a fondo, rigurosa y formal, diseñada para identificar problemas tan cerca de su punto de origen como sea posible
- Medición: Es el proceso por medio del cual se miden la mayor cantidad de atributos del producto y proceso, con el fin de tener información cuantificable útil para mejora continua.
- Administración de la configuración

Validación ¿Estamos construyendo el producto correcto?

La validación busca asegurar que el sistema software satisface las expectativas del cliente. Es decir, va más allá de la comprobación de la funcionalidad del sistema.

Es el proceso de evaluar el software al final del proceso de desarrollo para asegurar que cumple con los requerimientos.

Las actividades de validación se dan después de que el software ha sido desarrollado, para determinar si se han implementado correctamente los requerimientos.

La validación comprende actividades dentro de tres procesos básicos:

- Prueba
- Medición
- Aumento de la confiabilidad del software
 - Ayuda en la toma de decisiones con respecto a cuando hay que dejar de probar y liberar el software

Tiene lugar en todas las etapas del proceso de desarrollo

Durante y después del proceso de implementación se debe realizar validación y verificación de software, es decir, el programa que se está desarrollando debe ser comprobado para asegurar que satisface su especificación y entrega la funcionalidad esperada por las personas que pagan por el software

Verificación y validación

Existen dos aproximaciones para el análisis y comprobación de los sistemas:

- Inspecciones de software: analizan y comprueban las representaciones del sistema
 - Técnicas de inspección
 - Inspección de programas
 - Análisis automático del código
 - Verificación formal
- Pruebas del software: implican ejecutar una implementación del software con datos de prueba.
 - Pruebas de validación
 - Intentan demostrar que el software es el que el cliente quiere
 - Pruebas de defectos
 - Intentan revelar defectos en el sistema en lugar de simular su uso operacional.

Depuración

Los procesos de validación y verificación intentan establecer la existencia de defectos en el sistema de software

La depuración es el proceso de localizar y corregir errores.

Luego de la depuración, debe volverse a inspeccionar o hacer las pruebas de regresión.

Planificación de pruebas de Validación y Verificación de software

Es necesaria una planificación para obtener el máximo provecho de las inspecciones y pruebas.

Se comienza la planificación en etapas tempranas del proceso de desarrollo

Modelo en V

Estructura de un plan de pruebas

- El proceso de prueba
- Trazabilidad de los requerimientos
- Elementos probados
- Calendario de pruebas
- Procedimientos de registros de las pruebas
- Requerimientos de hardware y software
- Restricciones

Inspecciones de software

¿Qué es la inspección de programas?

Son parte del proceso de verificación del software.

Las revisiones o inspecciones de software, son un proceso de v y v estático en el que un sistema software se revisa para encontrar errores, omisiones y anomalías sin examinar por qué surgieron ni como se remueven. Generalmente se centran en el código fuente, pero pueden aplicarse a cualquier producto del proceso de desarrollo.

Estas inspecciones toman lugar en puntos específicos de la línea de ensamblaje, y son usadas para asegurar que las partes y los ensamblajes están correctamente contruidos acorde a las especificaciones.

Son usadas para asegurar la calidad durante el proceso de desarrollo, no solo en la fase de implementación.

Si una organización realiza estadísticas sobre las inspecciones, tendrá un promedio de errores localizados y podrá indicar cuando un producto esta listo para pasar a la siguiente etapa del proceso de desarrollo.

Ventajas: Al ser estático no hay que preocuparse de las interacciones entre errores, y se pueden inspeccionar versiones incompletas de un sistema sin costos adicionales. También pueden verificarse si se cumplen atributos de calidad como portabilidad, mantenibilidad y cumplimiento de estándares.

Nombre y detalle los pasos de la inspección de programas

Proceso de inspección

- Planificación: se asignan los roles a los inspectores, se distribuyen las copias del producto y el material relacionado y se planifica la reunión de inspección.
- Conferencia Introductoria: realiza una presentación general del proyecto en desarrollo a los inspectores.
- Preparación: los inspectores se instruyen para la reunión de inspección estudiando el material aportado en la planificación.
- Reunión: los inspectores se reúnen. Se investiga el producto y se clasifican y registran los defectos encontrados. Al final los inspectores deciden si el trabajo es aceptado, retrabajado bajo la supervisión del moderador o para una próxima inspección. De esta reunión surge una lista de defectos encontrados y un reporte para la gerencia indicando producto inspeccionado, equipo de inspección, numero de defectos encontrados y su gravedad.
- Retrabajo: el autor del trabajo corrige los defectos detectados en la reunión de inspección.
- Seguimiento: las correcciones realizadas por el autor son chequeadas por el moderador, si este queda satisfecho la inspección concluye.

Roles

- Moderador: dirige la reunión de inspección, es el inspector mas experimentado. Asegura que el equipo esta preparado, mantiene le orden durante la reunión y focaliza el proceso de inspección en los defectos.
- Registrador: registra la localización y descripción de los defectos encontrados.
- Lector: presenta el producto durante la reunión.
- Autor: quien elaboro el producto.

Tipos de inspecciones o revisiones de software

Pueden ser informales como una reunión informal donde se discuten aspectos técnicos, o formales como una presentación formal del diseño del software ante un auditorio de clientes, gestores y personal técnico.

Las más comunes son:

- Auditorias: Técnica de verificación realizada durante el proceso de desarrollo de un nuevo producto o durante el mantenimiento.
Consiste en asegurar que el producto se ajusta a los planes, políticas, procedimientos y estándares establecidos.
Se realiza a través de reuniones, observaciones y examinando los productos y procedimientos.
- Auto - inspección: revisión intensa de un producto del trabajo para asegurar que esta correcto, completo, que es consistente y claro. Se realiza por alguien no involucrado en el desarrollo del producto.
- Inspecciones técnicas formales: Se realizar por un equipo constituido por moderador, secretario, lector, productor, agente de v y v y expertos, que examina un producto determinado siguiendo roles y un proceso bien definido.
- Revisiones: evalúa el producto en base a guías, estándares y especificaciones de desarrollo y provee a los administradores evidencia de que el proceso se está realizando acorde con los objetivos establecidos.
- Recorridos: su objetivo es detectar y documentar fallas, realizado por todos los involucrados (coordinador, presentador, secretario, encargado de mantenimiento, encargado de estándares, agente de acreditación que representa al usuario y revisores adicionales).

Tipos de inspección (ejemplos del ppt 04):

- Inspección de requerimientos: ¿los req de la ERS son consistentes y rastreables con respecto a la documentación que les precede? ¿son claros, concisos, consistentes, pueden ser probados?
- Inspección del diseño: ¿el diseño de la DDS aborda todos los req de la ERS? ¿son los elementos de diseño rastreables a requerimientos de la ERS?
- Inspección del código: ¿Es consistente el código con el diseño de la DDS?
- Inspección del procedimiento de pruebas: ¿las pruebas reflejan los req de la ERS?

Estrategias de pruebas de software

Estrategia de prueba

¿Qué es una estrategia de prueba?

Una estrategia de prueba integra las técnicas de casos de prueba en una serie de pasos bien planificados que dan como resultado una correcta construcción del software.

Debe incorporar la planificación de prueba, diseño de los casos de prueba, ejecución de las pruebas y la evaluación de los resultados

Son un conjunto de actividades que se pueden planificar por adelantado y llevar a cabo sistemáticamente.

Características:

- Comienzan a nivel de modulo y trabajan hacia la integración de todo el sistema.
- Son apropiadas diferentes técnicas de prueba.
- Las lleva adelante el responsable del proyecto o de un equipo independiente.

¿En qué momento termina el proceso de prueba?

La prueba nunca termina, ya que el responsable del desarrollo del software carga o pasa el problema al cliente.

Se termina la prueba cuando se agota el tiempo o el dinero disponible para tal afecto.

Se pueden desarrollar directrices para saber cuando se termina la prueba mediante la agrupación de métricas durante la prueba de software y haciendo uso de modelos de fiabilidad del software existente.

Implementación de estrategias de prueba

- Especificar los requisitos del producto de manera cuantificable mucho antes de que comiencen las pruebas.
- Establecer los objetivos de la prueba de manera explícita.
- Construir un software robusto diseñado para probarse a sí mismo.
- Usar revisiones técnicas formales efectivas como filtro antes de las pruebas.
- Llevar revisiones técnicas formales para evaluar la estrategia de prueba y los propios casos de prueba.
- Desarrollar un enfoque de mejora continua al proceso de prueba.

Pruebas:

- Prueba de unidad: se centra en verificar la menor unidad de diseño del software, el módulo.

Procedimiento:

- Se prueba la interfaz del modulo para asegurar que la información fluye de forma adecuada hacia y desde la unidad de programa que está siendo probada
- Se examinan las estructuras de datos locales para asegurar que los datos que se mantienen temporalmente conservan su integridad durante todos los pasos de ejecución del algoritmo.

- Se prueban las condiciones limite para asegurar que el modulo funciona correctamente en los limites establecidos como restricciones de procesamiento.
 - Se ejercitan todos los camino independientes de la estructura de control con el fin de asegurar que todas las sentencias del modulo se ejecutan por lo menos una vez.
 - Finalmente, se prueban todos los caminos de manejo de errores.
- Prueba de integración: se debe integrar los módulos para formar el paquete de software completo y probar. Técnica sistemática para construir la arquitectura del software, mientras se aplican las pruebas para descubrir errores asociados con la interfaz. Todos los componentes superaron las pruebas unitarias.

Tipos:

- Descendente: se integran los módulos moviéndolos hacia abajo por la jerarquía de control.
- Ascendente: Desde los módulos mas atómicos y suben en la jerarquía.

¿Qué es el testing de regresión?

- Pruebas de regresión: volver a ejecutar un subconjunto de pruebas que se han llevado a cabo anteriormente para asegurarse que los cambios no han propagado efectos colaterales no deseados.

- Pruebas en la OO:

Pruebas de unidad:

- Clases
- Métodos de las clases
- Se orienta mediante operaciones que encapsulas la clase y el comportamiento de estado de la clase

Subprocesos:

- Conjunto de clases requeridas para responder a un evento o entrada
- Cada subproceso se prueba independientemente

Basada en uso

- Se prueban las clases independientes, luego las clases dependientes de estas.

¿Cuál es el propósito de las pruebas de validación?

¿Qué resultados da este proceso?

- Prueba de validación: Se concentra en las acciones visibles para el usuario y en la salida que este puede reconocer.

Es una serie de pruebas que demuestran que se cumplen con los requisitos.

En el plan de pruebas ya esta establecido las clases de pruebas, el procedimiento y los casos de prueba

Resultados

- Las características cumplen con la especificación y se acepta
- Se descubre una desviación y se crea una lista de deficiencias.

Explique cuál es la estrategia de prueba que generalmente utilizan la mayoría de los equipos de prueba de software.

Explique pruebas alfa y beta

Pruebas alfa y beta: se utiliza para descubrir errores que parezcan que solo el usuario final puede descubrir.

Prueba alfa: se lleva a cabo por un cliente en el lugar de desarrollo. Se usa el software de manera natural con el desarrollador como observador del usuario y registrando los errores y los problemas de uso.

Prueba beta: se llevan a cabo por los usuarios finales del software en los lugares de trabajo de los clientes. El desarrollador no está presente. Es una aplicación en vivo del software en un entorno que no puede ser controlado por el desarrollador. El cliente informa de los errores al desarrollador.

- Prueba de sistema: verifica que cada elemento encaja de forma adecuada y se alcanza la funcionalidad y el rendimiento del sistema total. Pruebas cuyo propósito es ejercitar profundamente el sistema de computo
 - *Pruebas de recuperación*: fuerza el fallo del software de muchas formas y verifica que la recuperación se lleve a cabo apropiadamente.
 - *Pruebas de seguridad*: intenta verificar que los mecanismos de proyección incorporados en el sistema lo protegerán de accesos impropios.
 - *Prueba de resistencia*: ejecuta un sistema de forma que demande recursos en cantidad, frecuencia o volúmenes anormales.
 - *Prueba de desempeño*: esta diseñada para probar el rendimiento del software en tiempo de ejecución dentro del contrato de un sistema integrado.

Técnicas de pruebas de software

Fundamentos de las pruebas

El ingeniero de software crea una serie de casos de prueba que intentan demoler el software construido.

¿Cuáles son los objetivos de las pruebas?

Objetivos de las pruebas

- La prueba es el proceso de ejecución de un programa con la intención de descubrir un error.
- Un buen caso de prueba es aquel que tiene una alta probabilidad de demostrar un error no descubierto hasta entonces.
- Una prueba tiene éxito si descubre un error no detectado hasta ahora.
- Diseñar pruebas que saquen a la luz diferentes clases de errores, haciéndolo en la menor cantidad de tiempo y esfuerzo.
- Los datos que proporcionan las pruebas son un buen indicador de fiabilidad del software.
- Las pruebas no demuestran la falta de defectos, sino la presencia de estos.

Principios de las pruebas

- A todas las pruebas se les debería hacer un seguimiento hasta los requisitos del cliente.
- Las pruebas deberían planificarse antes de que comiencen.
- Principio de **Pareto** aplicable a las pruebas.
- Empezar por lo pequeño y progresar hacia lo grande.
- No es posible las pruebas exhaustivas.
- Las pruebas deberían ser realizadas por un equipo independiente.

Atributos de una buena prueba

- Tiene una alta probabilidad de encontrar un error.
- No debe ser redundante.
- Debería ser la mejor de su clase.
- No debe ser demasiado sencilla ni demasiado compleja.

Explique los enfoques del diseño de casos de prueba.

Enuncie dos técnicas de cada uno de los enfoques de casos de prueba.

Diseño de casos de prueba

- Diseñar pruebas que tengan la mayor probabilidad de encontrar el mayor numero de errores con la misma cantidad de esfuerzo y tiempo posible.

Pruebas de caja blanca: conocen el funcionamiento del producto. Utilizan la estructura de control del diseño procedimental para obtener los casos de prueba.

Se obtienen casos de prueba que:

- Garantiza que se ejecutan por lo menos una vez todos los caminos independientes.
- Ejecuten todas las decisiones lógicas en sus vertientes verdades y falsas.
- Ejecuten todos los bucles en sus límites.
- Ejecuten las estructuras internas de datos.

¿Qué es prueba de camino básico?

Camino básico: técnica de prueba de caja blanca, permite obtener una medida de la complejidad lógica de un diseño procedimental y usar esa medida como guía para la definición de un conjunto básico de camino de ejecución.

Los casos de prueba obtenidos garantizan la ejecución de al menos una vez cada sentencia del programa.

Grafo del programa: representa el flujo de control lógico mediante una notación.

Complejidad ciclomática: Es una métrica de software que proporciona una medición cuantitativa de la complejidad lógica de un programa, el valor calculado define el numero de caminos independientes del conjunto básico de un programa y nos da un numero superior para el numero de pruebas que se deben realizar para asegurar que se ejecuta cada sentencia al menos una vez.

Camino independiente: constituido por lo menos por una arista que no haya sido recorrida anteriormente a la definición del camino.

Pruebas de estructura de control:

- Pruebas de condición
- Prueba de flujo de datos
- Pruebas de bucle

Pruebas de caja negra: conocen la función específica. Se centran en los requisitos funcionales del software. Permiten obtener un conjunto de condiciones de entrada que ejercitan completamente todos los requisitos funcionales de un programa. Intenta descubrir errores diferentes a caja blanca.

Errores:

- Funciones incorrectas o faltantes.
- Errores de interfaz.
- Errores en estructurado de datos o en acceso a base de datos.
- Errores de rendimiento.
- Errores de inicialización y terminación.

Partición equivalente:

- Método de prueba que divide el campo de entrada de un programa en clases de datos de los que se pueden derivar casos de prueba.
- Un caso de prueba ideal descubre de forma inmediata una clase de errores.
- Una clase de equivalencia representa un conjunto de estados validos o no validos para condiciones de entrada.

Directrices:

- Si la condición de entrada es un rango: se define una clase equivalencia valida y dos no válidas.
- Si requiere un valor específico: se define una clase de equivalencia valida y dos no válidas.
- Si especifica un miembro de un conjunto: se define una clase de equivalencia valida y una no válida.
- Si una condición de entrada es lógica, se define una clase de equivalencia valida o una no válida.
- Si se especifica un rango delimitado por a y b: diseñar casos de prueba para los valores a y b, para los valores justo por debajo y justo por encima de a y b.

- Si especifica un número de valores: diseñar casos de prueba que ejecuten los valores máximos y mínimos.
- Si las estructuras de datos internas tienen límites preestablecidos: asegurarse de diseñar un caso de prueba que ejecute la estructura de datos en sus límites.

Casos de prueba

¿Qué es un caso de prueba?

Un caso de prueba es un conjunto de acciones con resultados y salidas previstas basadas en los requisitos de especificación del sistema.

Componentes:

- Propósito de la prueba o descripción del requisito que se está probando
- Método o forma como se probará
- Versión o configuración de la prueba, versión de la aplicación en prueba, el hardware, el software, el sistema operativo, los archivos de datos, etc.
- Resultados, acciones y resultados esperados o entradas y salidas.
- Documentación de la prueba y sus anexos.

Pruebas orientadas a objetos

Estrategia:

Comenzar con probar lo pequeño y funciona hacia fuera, hacia lo grande.

- **Pruebas de unidad:** la unidad mínima de prueba es la clase, nombre, atributos, métodos.
Las pruebas de clase se conducen mediante las operaciones encapsuladas en la clase y su comportamiento.
 - *Casos de prueba:* cada caso de prueba debe ser identificado separadamente, y explícitamente asociado con la clase a probar.
Debe declararse el propósito de la prueba.
Debe desarrollarse una lista de pasos a seguir.
Secuencia de operaciones a ejecutar.
 - *Pruebas a nivel de clases:* se busca encontrar situaciones que rompan los supuestos que dan validez a la pieza de software que se está probando.
Además de la función que debe satisfacer cada método, se tiene el supuesto de que el estado es consistente sin importar el orden de ejecución de los métodos.
Establecer secuencias de prueba al azar.
Técnica: particiones a nivel de clases:
 - Cada atributo se relaciona solo con un conjunto de las operaciones de la clase.
 - Reduce el numero de casos de prueba para validar la clase.
 - Criterio: partición basada en estados.
 - Diagrama de transición de estados
- Separar las clases en partes (rebanadas): para cada atributo identificar las operaciones relacionadas a este.
- Generar rebanadas:
 - Grafo de llamadas: conjunto de árboles donde la raíz es un método y las hojas los atributos a los que hace referencia y métodos que invoca.
 - Matriz de uso mínima: una fila por cada atributo, una columna por cada método, las celdas según la relación entre atributo – método transforma, reporta, opera
 - Secuencia de métodos: se categorizan los métodos en constructores, reporteros, transformadores y otros.
 - Prueba de los módulos transformadores:
 - Se elige un atributo y los transformadores que corresponden a su rebanada colocando el reportero al final. Se forman las permutaciones de los transformadores, quedando el reportero siempre en el final.
 - Herencia: probar la clase base y las subclases derivadas con la misma estrategia.
 - Secuencia de métodos para la prueba
 - Generación de los casos de prueba
- **Pruebas de integración:**
 - Problemas de configuración
 - Funciones faltantes o conflictos
 - Uso incorrecto o inconsistentes de bd
 - Violaciones a la integridad de datos
 - Llamadas a método equivocados

- Parámetros erróneos
- Problemas del manejo de la máquina virtual
- Recursos insuficientes

Unidad mínima de funcionalidad: casos de uso

Se elaboran diagramas: colaboración / secuencia que representan los escenarios para una misma situación. Objetos que colaboran, interacciones entre objetos, secuencia de mensajes.

Toma importancia todos los componentes a integrar

De estos componentes interesa su estado expresado en proporciones lógicas.

Estado de los objetos involucrados: valores de los parámetros

Los valores que pueden recibir los métodos invocados aumentan los casos de prueba

Generar los casos de prueba: para cada diagrama de secuencia, generar los estados posibles de los elementos involucrados.

Para cada método en las interacciones incluidas en el diagrama de secuencia determinar los conjuntos de valores adecuados.

Con cada estado y cada representante de conjuntos de datos se forma un caso de prueba.

Combinando los diferentes valores de cada categoría.

Unir los componentes

Técnica sistemática para construir la arquitectura del software mientras se aplican las pruebas para descubrir errores asociados a la interfaz.

Cada componente probado se construye el programa según lo especificado en el diseño.

Integración descendiente: enfoque incremental para la construcción de la arquitectura de software. Empieza en el modulo principal y desciende a los módulos subordinados.

Enfoques:

Primero en profundizar

Primero en anchura

Verifica los principales puntos de control al principio del proceso de prueba

Puede traer problemas cuando los módulos inferiores realizar cálculos que son necesarios en módulos superiores.

Se utilizan módulos de resguardo

Integración ascendente: empieza la construcción y la prueba con módulos atómicos. Los de más bajo nivel. Se emplean controladores.

Enfoque combinado: integración descendente para los primeros niveles y pruebas ascendentes para los niveles subordinados.

La **SIT** se realiza principalmente para comprobar cómo los módulos individuales interactúan entre si cuando se integran en un sistema como un todo.

Se lleva a cabo después de la prueba unitaria y se realizara cada vez que se agregue un nuevo módulo al sistema.

Los casos de prueba de SIT se centran en la interfaz entre los componentes del sistema.

- **Pruebas de sistemas**: las pruebas del sistema se realizan principalmente para comprobar si todo el sistema funciona como se esperaba con referencia a los requisitos especificados.
 - Se lleva a cabo después de completar las pruebas de integración.
 - Los casos de prueba se centran en la simulación de escenarios de la vida real.
 - El software es incorporado a otros elementos del sistema y realizan una serie de pruebas de integración del sistema

- Se debe anticipar posibles problemas
 - Manejo de errores
 - Errores en la interfaz con otros componentes
 - Registrar los errores
- Una vez integrado completamente el sistema es posible probar propiedades emergentes, como el rendimiento y la confiabilidad.

Pruebas de recuperación: es una prueba del sistema que fuerza el fallo del software de muchas formas y verifica que la recuperación se lleva a cabo apropiadamente. Deben diseñarse para garantizar que el sistema procese su carga pretendida. Se hacen una serie de pruebas donde se aumenta la carga, hasta que el rendimiento del sistema se vuelve inaceptable.

Pruebas de seguridad: intenta verificar que los mecanismos de protección incorporados en el sistema lo protegerán de accesos impropios.

Pruebas de usabilidad: La usabilidad de una aplicación puede entenderse como una forma de medir cuanto de fácil es entender una aplicación para un usuario inexperto.

Sobre la interfaz podemos establecer reglas que unifiquen y homogeneicen los estilos, sin embargo, la usabilidad es algo más, ya que está ligada mas directamente al modo en que se realizan las operaciones del sistema y como de fácil resulte su uso para el usuario.

- **Pruebas de validación**: esta prueba es desde la perspectiva de los requisitos del usuario. Es realizado por clientes y usuarios finales.

Objetivos: demostrar que el software funciona de acuerdo a las expectativas del usuario.

La influencia del entorno de trabajo del usuario tiene un gran efecto sobre la fiabilidad, el rendimiento, el uso y la robustez de un sistema.

Pruebas de caja negra: demuestran la conformidad con los requisitos.

El plan de pruebas considera: casos de prueba para demostrar que se satisfacen los requisitos

No es posible prever como utilizara el software el futuro usuario

Pruebas de aceptación realizadas por el usuario

Pruebas alfa: se usa el software de forma natural con el desarrollador como observador del usuario y registrando los errores y los problemas de uso. Las pruebas alfa se llevan a cabo en un entorno controlado

Pruebas beta: se llevan a cabo por los usuarios finales del software en los lugares de trabajo de los clientes. El desarrollador no participa

Resultados: las características de funcionamiento o rendimiento están de acuerdo con las especificaciones y son aceptables.

Se descubre una desviación de las especificaciones y se crea una lista de deficiencias.

Métricas e indicadores objetivos para el control de proyectos de desarrollo de software

El problema

Es común que los proyectos de desarrollo de software terminen con importantes desvíos en costos y calendarios

Evidencia física

Los líderes de proyectos tenemos que evitar tomar decisiones subjetivas o a ojo

Para esto tenemos que tomar decisiones usando diferentes herramientas basadas en evidencia física

Una posible evidencia física que sirve como base para la registración de avance puede ser una porción de software construida.

¿Cuál es la evidencia? El producto desarrollado, probado y estabilizado

De esta manera estamos utilizando el resultado de control de calidad

¿Cuál es el grado de avance?

Avance por calendario: se mide avance solo por paso del tiempo

Avance por código completo: se mide avance cuando una porción del código esta completa, pero sin saber el grado de estabilidad que tiene.

El indicador de funcionalidad completa

El indicador de funcionalidad completa mide avance cuando una funcionalidad esta completa, pero no hay avance si la funcionalidad no esta completa, no esta completa la funcionalidad si no esta desarrollada, probada y estabilizada.

Paso 1: determinar las funcionalidades

Paso 2: asignar un peso a cada funcionalidad

Paso 3: estimar la fecha en que la funcionalidad estará completa

Paso 4: a medida que avanza el proyecto registrar las fechas reales de funcionalidad completa

Consideraciones:

Validez: este indicador solo es válido en etapa de construcción

Proceso: este indicador no generara avance si el equipo no se focaliza en cerrar temas

Síndrome del 0%: si se considera completa a una funcionalidad cuando no tienen ningún defecto, evitaremos el síndrome del 90%, pero caeremos en el síndrome del 0%

Funcionalidad = código: es su forma mas pura, este indicador solo considera funcionalidad al producto final para el usuario.

El indicador de nivel de calidad

El indicador de funcionalidad completa divide al producto en dos estados: funcionalidad completa o no completa.

Algunas veces necesitamos utilizar estados intermedios.

¿En qué fecha terminamos?

Con el indicador de funcionalidad completa podemos obtener una tendencia.

En la etapa de estabilización comienza cuando el indicador de funcionalidad completa deja de tener validez.

Solo resta corregir los defectos remanentes y encontrar los que no encontramos aun

Indicador de evolución de prueba

Se utiliza para obtener una tendencia y poder predecir la fecha de finalización

Mide los defectos, cuantos aparecen y cuantos se cierran

Proceso:

- Registrar defectos encontrados

- Registrar los cambios de estados

Consideraciones:

Validez: este indicador es valido durante todo el proyecto cuando hay prueba en paralelo al desarrollo

Proceso: el indicador es muy útil para detectar el síndrome del 90%

Indicador de cobertura de prueba

Muestra cuando habría que probar, cuanto se pudo probar y cuanto funciona bien.

Se realiza a partir de los estados de los casos de prueba (planificados, disponibles, ejecutados, ejecutados ok)

¿Cuánto gastaremos realmente?

Ya conocemos el grado de avance (indicador de funcionalidad completa y de nivel de calidad) y tenemos menos incertidumbre sobre la fecha de fin (indicador de evolución de la prueba y de cobertura de prueba). Ahora queremos saber cuanto costara realmente y usamos el indicador de valor agregado.

Indicador de valor agregado

La idea es usar el avance calculado por funcionalidad completa para calcular el valor agregado

Alcance: es necesario separar los costos que no están asociados directamente a la construcción de software.

Validez: al igual que el indicador de funcionalidad completa, solo aplica a la construcción, no a la estabilización.

Existe margen de error: los pesos generan información inexacta.

Métricas pruebas unitarias

Ofrece una medida de los componentes que se han probado individualmente antes de la integración con respecto al total de componentes que han sido implementados.

Exhaustividad de la prueba

Amplitud de cobertura de prueba: indicador de cuantos requisitos se han probado del número total de estos.

Profundidad de la prueba: porcentaje de los caminos básicos independientes probados

¿Cuál es el ciclo de vida del testing?

¿Cuáles son los pasos de ciclo de vida del testing?

¿Qué relación hay entre el plan de pruebas y el ciclo de vida del testing?

Explique qué actividades se hacen en cada uno de los pasos del ciclo de vida del testing.

09-CICLO DE VIDA DEL TESTING:

Planificación y control:

- Entendemos los objetivos del cliente, y los riesgos que se abordan con las pruebas. Las tareas de planificación nos ayudan a construir el plan de pruebas.
- Se determina el alcance, los riesgos y los objetivos de la prueba
- Se determina el método de prueba: considerando cómo vamos a llevar a cabo las pruebas, las técnicas a utilizar, quién debe involucrarse y cuando, y lo que vamos a producir como parte de las pruebas.
- Se debe asegurar la concordancia de las políticas y/o estrategias de prueba establecidas en la organización con lo que vamos a hacer.
- Se determinan los recursos de prueba necesarios: El equipo de prueba y el hardware y el software de apoyo que requerimos para el entorno de prueba.
- Se calendarizan en el plan de proyecto las actividades de prueba: El análisis, diseño, ejecución y la evaluación.
- Se determinan los criterios de salida: Criterio de cobertura que ayudan a llevar un registro para determinar si estamos completando las actividades de prueba correctamente y muestran las tareas y los controles que debemos completar antes de dar por finalizada la prueba.

Análisis y Diseño:

- Es la actividad en la que los objetivos generales de prueba se transforman en condiciones de prueba tangibles y en diseños de casos de prueba.
- Se revisa la base de pruebas: en busca de errores no detectados para evitar que lleguen al código. Además, se pueden bosquejar pruebas de caja negra.
- Se identifican las condiciones de prueba basados en el análisis de los elementos de prueba, sus especificaciones, y lo que sabemos acerca de su comportamiento y estructura.
- Se diseñan las pruebas: Utilizando técnicas apropiadas que ayuden a seleccionar los casos de prueba más representativos particulares al software que se quiere probar. Y se establece los procedimientos de prueba para los casos de prueba.
- Se diseña el entorno de prueba: Identificando la infraestructura y las herramientas necesarias.

Implementación y Ejecución:

- Implementación: Se desarrollan y priorizan los casos de prueba, a partir de estos se crea un conjunto de pruebas para su ejecución y se implementa el medio ambiente.
- Ejecución: Se ejecutan los casos de prueba registrando los resultados que se comparan con los esperados, luego se elabora un informe de discrepancias si las hubiera y se repiten las actividades de prueba como consecuencia de estas.

Evaluación de los criterios de salida y presentación de informes:

- Se hace para cada tipo de prueba, para determinar si se ha hecho lo suficiente para encontrar los errores: La suficiencia la determinan los criterios de salida de prueba y deben ser comprobados con los registros de prueba, evaluando si es necesario realizar más pruebas o si los criterios de prueba no eran correctos.

Actividades de cierre:

- Se recopilan los datos.
- Se verifica que todos los incidentes se hayan corregido.
- Se debe documentar los aspectos del proceso de prueba para su reutilización.
- Se entrega toda la documentación y software al área de soporte.
- Se evalúa el proceso de prueba para proyectos futuros.